

Panduan Dosen Lab 08 - Cloud Notes Next.js

Repo materi: [SeedFlora/meet8CloudService](#) · **dependensi lokal:** [Lab 06](#) pada host/Codespace yang sama · **durasi contoh:** 110 menit. Modul mahasiswa sudah memuat kunci challenge; tugas dosen adalah memandu observasi, memastikan setiap mahasiswa menjalankan kode sendiri, dan menguji alasan di balik status HTTP serta log.

Tujuan dan kasus kerja

Mahasiswa menjelaskan batas browser/server pada Next.js App Router, menguji GET/POST Route Handler, menyimpan catatan lewat API Compose/PostgreSQL, dan membedakan liveness dari readiness. Kasus kerja: staf melaporkan catatan gagal disimpan. Mereka harus membedakan validasi 400, API mati 503, dan data yang sudah tersimpan.

Prasyarat dan tata letak dua repo

1. Clone repo Lab 06 dan Lab 08 sebagai dua folder berdampingan. Di Codespaces, clone Lab 06 **ke dalam Codespace Lab 08** atau jalankan keduanya pada satu komputer. `localhost` pada dua Codespace yang terpisah menunjuk dua mesin berbeda.
2. Di root Lab 06 siapkan file `.env` dan secret sesuai README repo itu, lalu `docker compose up --build -d --wait`. Cek `docker compose ps` dan `http://127.0.0.1:8000/health`; API/DB/cache harus healthy. Jangan menampilkan isi secret di layar.
3. Di root Lab 08 pastikan Node ≥ 20.9 , jalankan `npm ci`, salin `.env.example` ke `.env.local`, dan pastikan `LOCAL_API_URL=http://127.0.0.1:8000`. Jalankan `npm run dev`, lalu buka `http://localhost:3000`. Browser dev pada `127.0.0.1:3000` dapat memicu proteksi origin HMR pada Next 16; URL `localhost` adalah jalur yang dipakai dalam praktik.
4. Buka tiga terminal: Compose Lab 06, Next.js Lab 08, dan perintah probe. Siapkan screenshot; contoh dalam modul berada di `screenshots/` repo ini.

Alur kelas menit demi menit

Menit	Dosen	Mahasiswa	Kriteria hasil
0-15	Jelaskan browser → Route Handler → API → DB/cache.	Gambar aliran dan tandai <code>page.tsx</code> , <code>route.ts</code> .	Variabel <code>LOCAL_API_URL</code> hanya pada server.
15-30	Jalankan Compose Lab 06 dan Next.	Ikuti dua terminal, cek status/ports.	API 8000 dan Next 3000 hidup.
30-45	Buka <code>/api/health</code> , <code>/api/ready</code> , lalu form.	Bandingkan liveness/readiness.	HTTP 200 dan JSON yang berbeda.
45-60	Kirim body tanpa judul serta JSON rusak.	Catat dua HTTP 400.	Tidak ada catatan baru.
60-75	Simpan catatan di browser dan via API.	Ambil screenshot, header request ID, log.	HTTP 201, GET 200, judul muncul.
75-90	Stop hanya API Lab 06; pulihkan.	Amati 200/503 dan <code>notes_proxy_unavailable</code> .	Data muncul kembali setelah start.
90-100	Jalankan <code>npm run check:local</code> .	Jelaskan 9 checks.	9 PASS, 0 FAIL.
100-110	Hentikan dev, typecheck/build, Git.	Simpan laporan dan push repo sendiri.	Build exit 0, staged file aman.

Kunci dan perintah demo

A. Liveness vs readiness. PowerShell: `Invoke-WebRequest http://localhost:3000/api/health -UseBasicParsing` dan `Invoke-WebRequest http://localhost:3000/api/ready -UseBasicParsing`. Bash: `curl -i` untuk kedua URL. Kunci: `health` menunjukkan `status=ok` walaupun backend tidak ada; `readiness` baru 200 jika `health` API Lab 06 berhasil. Lab 06 `/health` juga memeriksa PostgreSQL dan Redis. Ini relevan untuk health probe deployment dan insiden produksi.

B. Validasi. `curl -i http://localhost:3000/api/notes -H 'Content-Type: application/json' -d '{"content": "uji"}'` menghasilkan **400** dari Next karena judul tidak ada. Body `{}` juga 400 JSON tidak valid. Setelah keduanya, GET daftar tidak bertambah. UI memakai `required`, tetapi API tetap harus memvalidasi karena klien dapat mengirim request langsung.

C. Simpan dan korelasi. `curl -i -X POST http://localhost:3000/api/notes -H 'Content-Type: application/json' -d '{"title": "Audit API", "content": "Jalur lengkap berhasil"}'` menghasilkan **201** dan `X-Request-ID`. Baris log `notes_proxy` pada terminal Next membawa `requestId`, status, dan durasi. Cari ID yang sama; log sengaja tidak menyimpan isi catatan, password, atau token. GET `/api/notes` kembali **200** dan memuat judul. Form browser memakai Route Handler yang sama.

D. Insiden. Dari Lab 06, `docker compose stop api`. GET `/api/health` Next tetap 200; GET `/api/ready` 503; GET `/api/notes` 503 dan log `notes_proxy_unavailable`. `docker compose start api`, tunggu healthy, ulangi probe; `/api/ready` 200 dan data sebelumnya tampak. Hindari `docker compose down -v` karena itu menghapus volume latihan. Jangan ganggu stack bersama bila kelompok lain sedang menjalankan pemeriksaan.

E. Checker dan build. `npm run check:local` melakukan sembilan check: liveness, readiness, dua validasi 400, POST 201, header ID, isi objek, GET 200, dan catatan ditemukan. Ia menyimpan satu catatan uji dengan judul `Lab08-check-....`. `npm run typecheck` dan `npm run build` harus exit code 0. Jika port 3000 bentrok, hentikan service lama; jangan mengganti port tanpa menyesuaikan URL checker (`LAB08_BASE_URL`).

Lab 08 - Next.js App Router

Cloud Notes

Catatan kelas dengan dua jalur data: API Compose lokal atau Supabase Auth + RLS.

Mode aktif: **API lokal**

Tulis catatan

Judul

Isi

Simpan

Daftar catatan

Catatan praktik Lab 08
Browser ke Next.js Route Handler ke API Lab 06 ke PostgreSQL.

Lab08-check-1791376974359
Catatan uji jalur browser ke PostgreSQL.

Audit kelas Lab06
Bukti persistensi PostgreSQL dan cache Redis

Browser menampilkan catatan yang disimpan

Command/tindakan: `npm run dev`, isi form, klik **Simpan**. **Fungsi:** menguji jalur empat lapisan. **Cara kerja:** browser POST ke Next, proxy ke Lab 06, API menulis ke PostgreSQL, frontend GET ulang. **Baca hasil:** judul baru berada di daftar.

Pretty-print ☐

```
{"status":"ready","dependency":"lab06-api"}
```

Respons readiness nyata

Command/tindakan: buka `/api/ready`. **Fungsi:** melihat kesiapan dependensi. **Cara kerja:** server Next memanggil health API Lab 06 dengan timeout. **Baca hasil:** `status=ready`, HTTP 200; ketika API mati, HTTP 503.

Pretty-print ☐

```
{"status": "unavailable", "dependency": "lab08-api"}
```

Readiness saat API Lab 06 mati

Command/tindakan: `docker compose stop api` dari repo Lab 06, lalu buka `/api/ready`. **Fungsi:** demonstrasi kegagalan dependensi. **Cara kerja:** Next tetap hidup tetapi fetch health upstream gagal. **Baca hasil:** HTTP 503 dan `status=unavailable`; pulihkan API dan tunggu healthy sebelum lanjut.

Lab 06 · Next.js App Router

Cloud Notes

Catatan kelas dengan dua jalur data: API Compose lokal atau Supabase Auth + RLS.

Mode aktif: **API lokal**

Tulis catatan

Judul

Isi

Simpan

Error: API lokal gagal. Jalankan Compose Lab 06.

Daftar catatan

Belum ada catatan yang terlihat.

N

Pesan pengguna ketika daftar tidak dapat dimuat

Command/tindakan: refresh / selama API mati. **Fungsi:** menghubungkan sinyal teknis 503 dengan gejala pengguna. **Cara kerja:** Client Component fetch `/api/notes`, lalu menampilkan pesan error bila proxy gagal. **Baca hasil:** pesan API lokal gagal dan daftar tidak terisi; data tetap ada setelah pemulihan.

Lab 08 | keluaran perintah yang diuji

```
$ npm run check:local
```

```
> cloud-notes-web@0.8.0 check:local
> node tests/challenge.mjs
```

```
PASS liveness Next.js HTTP 200
PASS readiness API Lab 06 HTTP 200
PASS judul kosong ditolak HTTP 400
PASS JSON rusak ditolak HTTP 400
PASS POST catatan menghasilkan HTTP 201
PASS respons proxy membawa X-Request-ID
PASS catatan tersimpan dengan judul yang benar
PASS GET catatan menghasilkan HTTP 200
PASS catatan baru tampak dalam daftar
challenge: 9 PASS, 0 FAIL
```

Cuplikan output aktual; ditata ulang agar terbaca pada modul.

Hasil checker lokal Lab 08

Command: `npm run check:local` setelah pemulihan. **Fungsi:** gerbang akhir jalur browser/server/backend. **Cara kerja:** sembilan request/validasi dibuat otomatis oleh `tests/challenge.mjs`. **Baca hasil:** 9 PASS, 0 FAIL; cuplikan output aktual ditata ulang agar mudah dibaca.

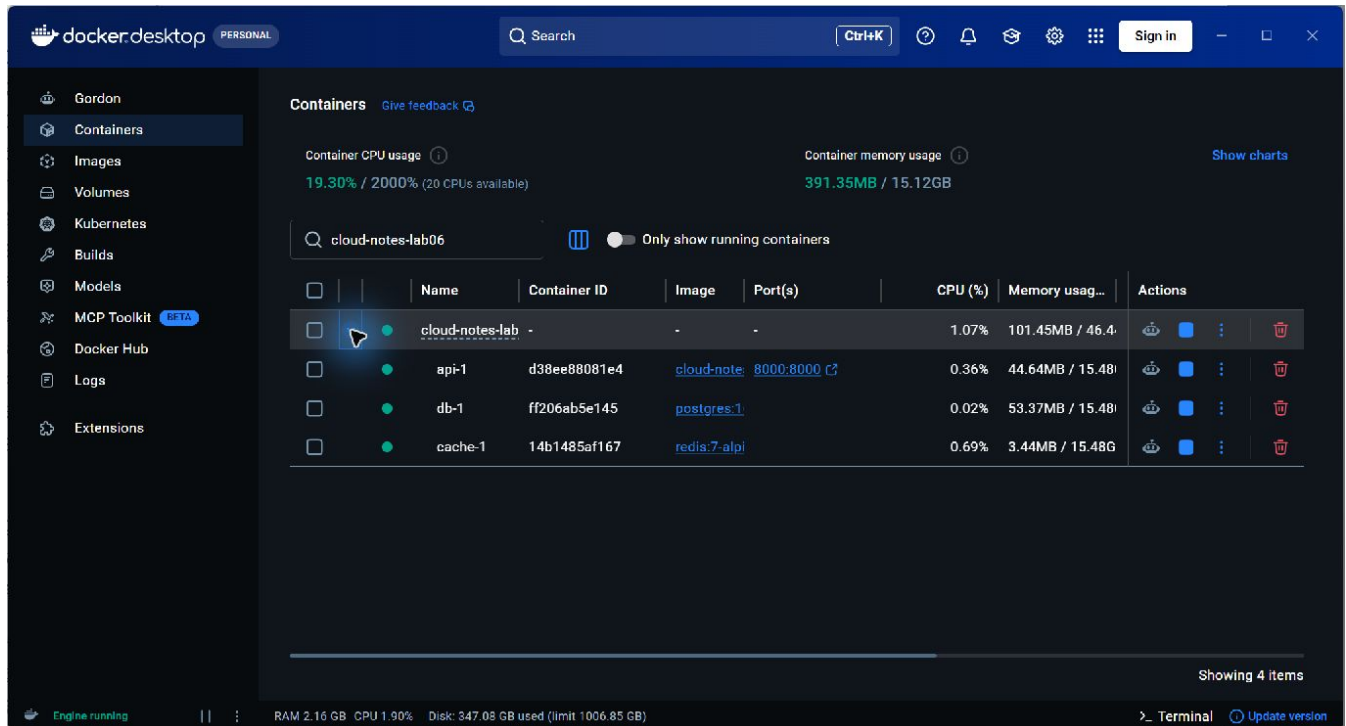
Kunci diskusi mahasiswa

1. `'use client'` diperlukan untuk state/form/event pada `page.tsx`; Route Handler berjalan di server dan boleh membaca `LOCAL_API_URL` privat.
2. Mematikan API tidak mematikan Next, jadi halaman dan health Next tetap hidup; readiness serta catatan gagal 503. Data PostgreSQL tetap ada.
3. Nilai `NEXT_PUBLIC_` terlihat oleh browser. Supabase publishable key dipakai bersama RLS; service role/secret key harus hanya di server.
4. Daftar catatan pribadi harus dimuat per pengguna/request. SSG/ISR bersama berisiko menampilkan data yang salah; SSR per request atau Client Component dengan auth/RLS sesuai konteks.

Rubrik, troubleshooting, dan keamanan

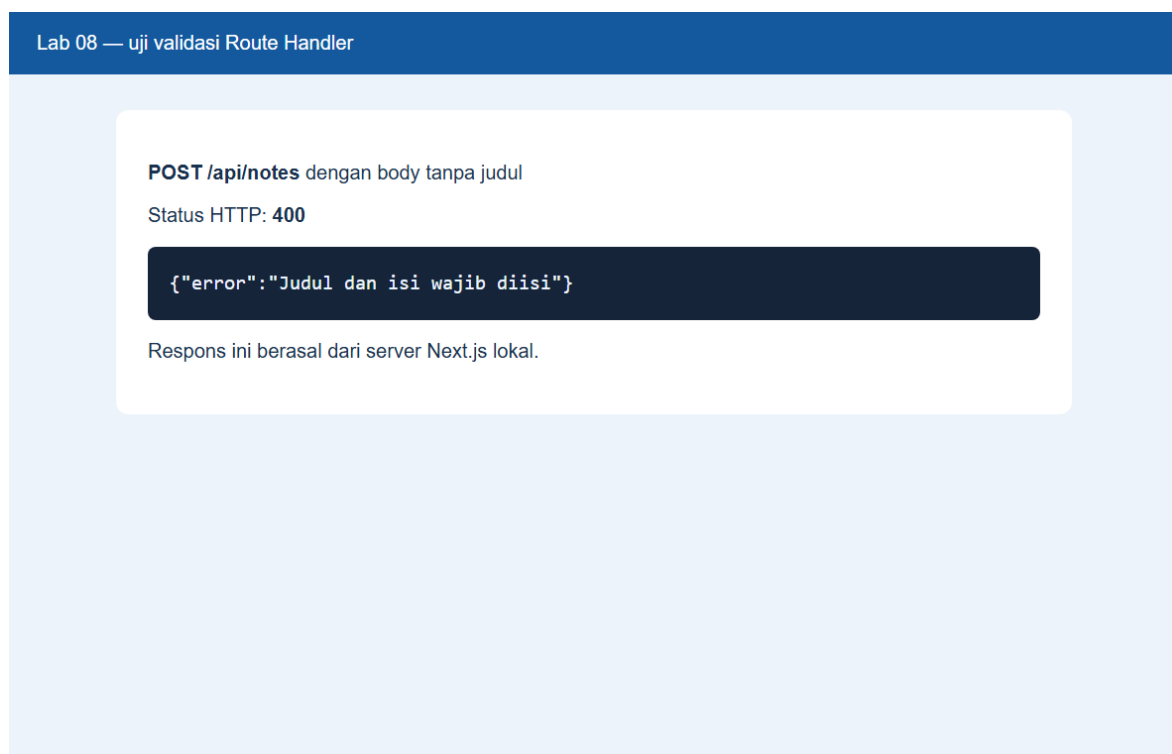
Nilai arsitektur/aliran (20%), request HTTP dan validasi (20%), simpan serta korelasi log (20%), insiden/pemulihan (20%), checker/build/Git (20%). Jika `npm run dev` menunjukkan halaman tetapi data kosong, periksa `/api/ready`, Lab 06 `/health`, dan `LOCAL_API_URL`. Jika browser UI tidak bereaksi di Next dev, pastikan URL **localhost** dan bukan 127.0.0.1 pada mesin uji ini. Jika checker gagal setelah API dipulihkan, tunggu `docker compose ps healthy` lalu ulangi. Periksa `git diff --cached --name-only` agar `.env.local`, `node_modules`, `.next`, dan secret tidak ikut. Mahasiswa push ke repo milik sendiri menggunakan [panduan Git](#).

Bukti visual tambahan untuk demo



Docker Desktop backend Lab 06

Command: `docker compose up --build -d --wait` dari repo Lab 06, lalu buka Docker Desktop. **Fungsi:** memeriksa prasyarat. **Cara kerja:** Compose menjalankan API, DB, dan Redis; API memetakan host 8000. **Baca hasil:** tiga service healthy.



Validasi HTTP 400 tanpa judul

Command: `POST /api/notes` dengan body hanya `content`. **Fungsi:** membedakan input salah dari outage. **Cara kerja:** Route Handler menolak payload sebelum proxy. **Baca hasil:** HTTP 400; respons aktual ditata dalam gambar agar terbaca.

Lab 08 — TypeScript dan build Next.js — keluaran aktual praktik lokal

\$ npm run typecheck

```
> cloud-notes-web@0.8.0 typecheck
> tsc --noEmit
```

\$ npm run build

```
✓ Compiled successfully in 285ms
  Running TypeScript ...
  Finished TypeScript in 1484ms ...
  Collecting page data using 6 workers ...
  Generating static pages using 6 workers (0/5) ...
  Generating static pages using 6 workers (1/5)
  Generating static pages using 6 workers (2/5)
  Generating static pages using 6 workers (3/5)
✓ Generating static pages using 6 workers (5/5) in 637ms
  Finalizing page optimization ...
```

Route (app)

```
└─ ○ /
└─ ○ /_not-found
└─ f /api/health
└─ f /api/notes
```

○ (Static) prerendered as static content
f (Dynamic) server-rendered on demand

Build Next.js lulus

Command: `npm run typecheck` lalu `npm run build`. **Fungsi:** gerbang source sebelum push. **Cara kerja:** TypeScript memeriksa tipe dan Next membuat bundle. **Baca hasil:** exit code 0; gambar dari run praktik sebelumnya, jalankan ulang pada mesin kelas.